

Orsus控制平台 API V1.3 (对外)

版本	修改时间	备注
V1.3	2026年7月1日	删除老版本兼容接口
V1.2	2026年6月9日	更新导航交互逻辑，补充接口信息
V1.1	2026年4月10日	
V1.0	2026年4月1日	

1. 概述

1.1 基础信息

项目	说明
默认端口	8898
API 前缀	/v1/api
健康探针	GET /healthz （不在 /v1/api 下）
WebUI	GET /

1.2 统一响应格式

普通 JSON API 使用统一响应体，业务状态通过响应体中的 `code` 字段区分。HTTP 状态码反映传输层/请求语义：

- 成功响应：HTTP 200，且 `code == 0`。
- 错误响应：HTTP 状态码按业务错误码映射，且 `code != 0`。
- 客户端必须同时处理 HTTP 状态码和响应体 `code`。`msg` 字段包含人类可读的错误描述，可直接用于界面提示。

成功响应（`code` 为 0）：

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "robot_type": "go2",
6      "connected": true,
7      "battery": 85
8    }
9  }
```

错误响应（code 非零）：

代码块

```
1  {
2    "code": 1001,
3    "msg": "invalid request body: scene_name is required",
4    "data": null
5  }
```

对应 HTTP 状态码：400 Bad Request。

代码块

```
1  {
2    "code": 2001,
3    "msg": "navigation container is not running",
4    "data": null
5  }
```

对应 HTTP 状态码：500 Internal Server Error。

代码块

```
1  {
2    "code": 1005,
3    "msg": "Downstream service unavailable",
4    "data": null
5  }
```

对应 HTTP 状态码：502 Bad Gateway。

传输型接口例外：

以下接口返回流、文件、WebSocket 握手、静态资源或 Swagger UI，不适用普通 JSON envelope 成功响应规则：

- SSE: GET /v1/api/systems/stream、POST /v1/api/interact/asr 成功响应。
- 文件下载: GET /v1/api/maps/{map_name}/files/{file_path}、GET /v1/api/maps/{map_name}/archive 成功响应。
- WebSocket: GET /v1/api/terminal/ws 成功握手。
- WebUI: GET /、GET /assets/* 等内嵌前端资源。

传输型接口在进入流式/下载/握手前发生参数、认证或上游错误时，仍尽量返回 JSON APIResponse。

字段	类型	说明
code	int	0 表示成功，非零为业务错误码（见下方错误码表）
msg	string	成功时为 "success"，失败时为具体错误描述
data	any null	业务数据，错误时通常为 null

1.3 业务错误码表

错误码	名称	说明
0	Success	操作成功
1001	BadRequest	请求参数错误
1002	NotFound	资源未找到
1003	Internal	内部服务错误
1004	NotImplemented	接口未实现（占位）
1005	Downstream	下游服务不可用
1006	Unauthorized	认证失败
2001	Navigation	导航服务错误
3001	Mapping	建图服务错误
4001	Containers	容器管理错误

4002	ContainerPolicyViolation	策略拒绝（互斥/保护策略不满足）
4003	ContainerStartTimeout	容器启动超时
4004	ContainerHealthCheckFailed	容器健康检查失败
4005	ContainerRuntimeUnreachable	Docker 运行时不可达
5001	Task	任务执行错误
6001	Maps	地图资源错误

业务码到 HTTP 状态码映射:

业务码	HTTP 状态码
0	200 OK
1001	400 Bad Request
1002	404 Not Found
1003	500 Internal Server Error
1004	501 Not Implemented
1005	502 Bad Gateway
1006	401 Unauthorized
4002	409 Conflict
2001 , 3001 , 4001 , 4003 , 4004 , 4005 , 5001 , 6001	500 Internal Server Error
未归类业务码	422 Unprocessable Entity

1.4 异步操作模式

以下操作为**异步执行**，接口立即返回成功，后台运行实际操作：

- 容器启动/停止（ /runtime/containers/{name}/actions/start|stop ）
- 导航容器停止（ /nav/container/stop ）

客户端应通过**轮询对应的状态查询接口**确认操作完成。

以下操作采用**快速失败检测模式**：接口提交任务后短暂等待（500ms ~ 5s），若任务在等待窗口内失败则立即返回错误，否则返回 task_id：

- 单点/路径导航启动 (/nav/start_navigation、 /nav/start_path_navigation)
- 建图容器启动 (/mapping/container/start)

2. 系统信息 (Systems)

GET /v1/api/systems/device

查询设备基础信息。数据来源于 `device_info.json` 与设备 Profile YAML 配置，并附加当前节点角色。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "SN": "DEVICE-001",
6      "model": "GS-Robot-X1",
7      "version": "v1.0.0",
8      "node_role": "worker",
9      ...
10   }
11 }
```

字段	类型	说明
SN	string	设备序列号（唯一标识）
model	string	机器人型号
version	string	设备版本号
node_role	string	当前节点角色（由配置文件指定）

data 的具体字段取决于 device_info.json 内容及设备 Profile 配置，上表为常见字段。

GET /v1/api/systems/health

查询系统健康状态与运行时长。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "ok",
6      "uptime_seconds": 3600,
7      "timestamp": "2026-04-10T10:00:00Z"
8    }
9  }
```

字段	类型	说明
status	string	健康状态，正常时为 "ok"
uptime_seconds	int	系统运行时长（秒）
timestamp	string	当前时间（ISO 8601 UTC）

GET /v1/api/systems/hub

查询边缘主机遥测数据（CPU、内存）。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "hostname": "edge-host-01",
6      "cpu_usage": 45.2,
7      "mem_used_mb": 4096,
8      "mem_total_mb": 8192
9    }
10 }
```

字段	类型	说明
hostname	string	主机名
cpu_usage	float	CPU 使用率（%，保留一位小数，200ms 采样）
mem_used_mb	int	已使用内存（MB）
mem_total_mb	int	总内存（MB）

数据通过读取 /proc/stat 和 /proc/meminfo 获取，非 Linux 平台返回 0。

GET /v1/api/systems/robot

查询机器人运行态信息。数据实时从运动控制下游服务（Motion）获取，包含机器人类型、连接状态和电量信息。

请求参数: 无

成功响应（适配器已连接）：

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "robot_type": "go2",
6      "connected": true,
7      "battery": 85
8    }
9  }
```

成功响应（多电池机器人）：

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "robot_type": "m20",
6      "connected": true,
7      "battery": [85, 90]
8    }
9  }
```

```
9 }
```

成功响应（运动控制服务不可达）：

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "robot_type": "unknown",
6      "connected": false,
7      "battery": null
8    }
9 }
```

字段	类型	说明
robot_type	string	当前激活的机器人适配器类型（如 go2 、 m20 ），无适配器时为 "unknown"
connected	bool	机器人是否已连接
battery	int int[] null	电池电量（%）。单电池为数字，多电池为数组，不可用时为 null

GET /v1/api/systems/stream

通过 Server-Sent Events (SSE) 实时推送系统状态事件。

Content-Type: text/event-stream

事件类型:

事件名	推送频率	说明
health	每 5 秒	系统健康状态快照
robot	每 10 秒	机器人聚合状态快照

连接建立后**立即推送一次当前值**，之后按频率定时推送。客户端断开时服务端自动清理。

SSE 数据格式:

代码块

```
1  event: health
2  data: {"status":"ok","uptime_seconds":3600,"timestamp":"2026-04-10T10:00:00Z"}
3
4  event: robot
5  data: {"robot_type":"go2","connected":true,"battery":85}
```

详见 附录 B: SSE 事件流协议。

3. 容器管理 (Containers)

管理设备上运行的容器化服务（AI 模型推理、基础设施服务等）。

GET /v1/api/runtime/containers/models

获取所有已注册的容器模型服务名称列表。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": ["llm-service", "vlm-service", "asr-service"]
5  }
```

GET /v1/api/runtime/containers

查询容器服务列表，支持按分类过滤。

查询参数:

参数	类型	必填	说明
class	string	否	服务分类过滤，可选值: model、infra

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": [
5      {
6        "name": "llm-service",
7        "service_class": "model",
8        "role": "language-model",
9        "container_name": "gs-llm",
10       "state": "HEALTHY",
11       "health": "healthy",
12       "critical": true,
13       "allow_manual_stop": false,
14       "mutex_group": "gpu"
15     }
16   ]
17 }
```

ContainerInfo 字段说明:

字段	类型	说明
name	string	容器服务名称（唯一标识）
service_class	string	服务分类: model / infra
role	string	服务角色描述
container_name	string	Docker 容器名
state	string	容器状态: UNKNOWN / STOPPED / HEALTHY / DEGRADED / FAILED
health	string	健康检查结果
critical	bool	是否为关键服务
allow_manual_stop	bool	是否允许手动停止
mutex_group	string	互斥组（同组服务不可同时运行，如 GPU 资源）

GET /v1/api/runtime/containers/{name}

查询指定容器服务的详细信息。

路径参数:

参数	类型	必填	说明
name	string	是	容器服务名称

成功响应: 同 ContainerInfo 结构（见上方字段说明）。

POST /v1/api/runtime/containers/{name}/actions/start

启动指定容器服务。**异步操作**——立即返回，后台执行（可能涉及 GPU 初始化，耗时较长）。

路径参数:

参数	类型	必填	说明
name	string	是	容器服务名称

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

操作后台执行，超时上限 5 分钟。客户端应轮询 GET /runtime/containers/{name} 确认 state 变为 HEALTHY。

POST /v1/api/runtime/containers/{name}/actions/stop

停止指定容器服务。**异步操作**——立即返回，后台执行。

路径参数:

参数	类型	必填	说明
name	string	是	容器服务名称

请求体（可选）：

代码块

```
1  {
2    "force": false
3  }
```

字段	类型	必填	默认值	说明
force	bool	否	false	是否强制停止（跳过优雅退出）

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

客户端应轮询 GET /runtime/containers/{name} 确认 state 变为 STOPPED。

4. 下游服务控制 (Services)

Orsus 统一代理三个下游子服务的控制接口，客户端通过 Orsus 网关调用，无需直连子服务端口。Orsus 内部会转发请求到对应子服务并将响应归一化后返回。

架构概览:

代码块

```
1  客户端 → Orsus (:8899/v1/api/services/*)
2           |→ 定位扫描服务 (Scan, 默认 :8080)
```

- 3

└─> 语义建图服务 (Semantic Map, 默认 :9000)
- 4

└─> 运动控制服务 (Motion / Robot Sport, 默认 :9098)

归一化状态说明: Orsus 将各子服务的异构状态统一映射为以下枚举值：

归一化状态	说明
running	服务正常运行中
stopped	服务已停止
degraded	服务部分可用（如定位扫描中部分子服务异常）
unknown	无法确定状态（通常为子服务不可达）

GET /v1/api/services/status

并发查询所有下游服务状态汇总。三个子服务的状态查询并行执行，任一子服务不可达时其状态返回 `unknown`。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "scan": {
6        "status": "running",
7        "detail": {
8          "services": [
9            { "name": "websocket-server", "type": "WEBSOCKET_SERVER", "state":
"RUNNING" },
10           { "name": "gs-receiver", "type": "GS_RECEIVER", "state": "RUNNING" },
11           { "name": "sensors-tower", "type": "SENSORS_TOWER", "state":
"RUNNING" }
12         ]
13       }
14     },
15     "semantic": {
16       "status": "stopped",
17       "detail": {
```

```
18         "status": "stopped",
19         "timestamp": 1712736000
20     }
21 },
22 "motion": {
23     "status": "running",
24     "detail": {
25         "state": "CONNECTED",
26         "active_adapter": "go2",
27         "busy": false
28     }
29 }
30 }
31 }
```

ServiceStatus 字段说明:

字段	类型	说明
status	string	归一化状态: running / stopped / degraded / unknown
detail	object	子服务原始状态详情（各服务结构不同，见各子服务节）

4.1 定位扫描 (Scan)

定位扫描服务管理三个子组件的生命周期：

子服务名	类型标识	说明
websocket-server	WEBSOCKET_SERVER	WebSocket 数据推送服务
gs-receiver	GS_RECEIVER	GS 数据接收器
sensors-tower	SENSORS_TOWER	传感器塔采集服务

此外还提供 data_send 组合命令，按依赖顺序批量启停 sensors-tower 和 gs-receiver。

GET /v1/api/services/scan/status

查询定位扫描服务归一化状态。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "running",
6      "detail": {
7        "services": [
8          {
9            "name": "websocket-server",
10           "type": "WEBSOCKET_SERVER",
11           "state": "RUNNING"
12         },
13         {
14           "name": "gs-receiver",
15           "type": "GS_RECEIVER",
16           "state": "STOPPED"
17         },
18         {
19           "name": "sensors-tower",
20           "type": "SENSORS_TOWER",
21           "state": "RUNNING"
22         }
23       ]
24     }
25   }
26 }
```

归一化规则:

子服务状态组合	归一化结果
全部 RUNNING	running
全部 STOPPED	stopped
部分运行部分停止	degraded
服务不可达	unknown

子服务状态字段说明:

字段	类型	说明
name	string	子服务名称
type	string	子服务类型标识
state	string	子服务运行状态: RUNNING / STOPPED

POST /v1/api/services/scan/start

启动全部定位扫描子服务。等价于调用下游 data_send/start 组合命令——按 sensors-tower → gs-receiver 顺序依次启动。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "success": true,
6      "service_name": "data_send",
7      "operation": "start",
8      "message": "数据发送服务启动成功",
9      "results": [
10     {
11       "success": true,
12       "service_name": "sensors-tower",
13       "operation": "start",
14       "message": "传感器塔启动成功"
15     },
16     {
17       "success": true,
18       "service_name": "gs-receiver",
19       "operation": "start",
20       "message": "GS接收器启动成功"
21     }
22   ]
23 }
```



```
24 }
```

错误: 1005 — 定位扫描服务不可达。

幂等性: 若子服务已处于目标状态, 返回 `skipped: true` 且不报错。

POST /v1/api/services/scan/stop

停止全部定位扫描子服务。按 `gs-receiver` → `sensors-tower` 逆序依次停止。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "success": true,
6      "service_name": "data_send",
7      "operation": "stop",
8      "message": "数据发送服务停止成功",
9      "results": [
10     {
11       "success": true,
12       "service_name": "gs-receiver",
13       "operation": "stop",
14       "message": "GS接收器已停止"
15     },
16     {
17       "success": true,
18       "service_name": "sensors-tower",
19       "operation": "stop",
20       "message": "传感器塔停止成功"
21     }
22   ]
23 }
24 }
```

错误: 1005 — 定位扫描服务不可达。

POST /v1/api/services/scan/{name}/start

启动指定的扫描子服务。

路径参数:

参数	类型	必填	说明
name	string	是	子服务名称: websocket-server / gs-receiver / sensors-tower / data_send

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "success": true,
6      "service_name": "sensors-tower",
7      "operation": "start",
8      "message": "传感器塔启动成功",
9      "remote_response": {
10       "code": 0,
11       "msg": "success"
12     }
13   }
14 }
```

当 name 为 data_send 时，按组合顺序启动 sensors-tower → gs-receiver，响应含 results 数组。

下游响应字段说明:

字段	类型	说明
success	bool	操作是否成功
service_name	string	子服务名称
operation	string	操作类型: start / stop
message	string	结果描述

skipped	bool	若服务已处于目标状态则为 true（幂等）
remote_response	object	远程服务（如 sensors-tower）的原始响应
results	array	仅 data_send 组合命令时出现，各子服务操作结果数组

错误: 1005 — 定位扫描服务不可达。

POST /v1/api/services/scan/{name}/stop

停止指定的扫描子服务。

路径参数:

参数	类型	必填	说明
name	string	是	子服务名称: websocket-server / gs-receiver / sensors-tower / data_send

成功响应: 结构同 start，operation 字段为 "stop"。

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "success": true,
6      "service_name": "gs-receiver",
7      "operation": "stop",
8      "message": "GS接收器已停止"
9    }
10 }
```

错误: 1005 — 定位扫描服务不可达。

4.2 语义建图 (Semantic)

语义建图服务负责场景语义数据的采集与存储。服务本身不要求请求体参数，所有接口均为无参调用。

幂等性: 重复调用 start（已运行时）或 stop（已停止时）均返回成功，不会报错。

GET /v1/api/services/semantic/status

查询语义建图服务归一化状态。

请求参数: 无

成功响应（服务停止时）：

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "stopped",
6      "detail": {
7        "status": "stopped",
8        "timestamp": 1712736000
9      }
10   }
11 }
```

成功响应（服务运行中）：

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "running",
6      "detail": {
7        "status": "running",
8        "timestamp": 1712736000,
9        "statistics": {
10         "total_images_saved": 128,
11         "total_image_errors": 0,
12         "async_processing": {
13           "containers_processed": 64,
14           "containers_dropped": 0,
15           "messages_dropped": 0,
16           "dedup_storage_running": true,
```

```
17         "queue_size": 3,
18         "max_queue_size": 100
19     },
20     "config": {
21         "debug_mode": false
22     },
23     "client_manager": {
24         "running": true,
25         "connected": true,
26         "total_messages_received": 500,
27         "total_messages_processed": 500,
28         "total_files_saved": 128,
29         "total_processing_errors": 0,
30         "total_messages_dropped": 0,
31         "current_queue_size": 2,
32         "queue_high_watermark": 15,
33         "queue_usage_percent": 2.0,
34         "avg_queue_wait_time_ms": 5.2,
35         "avg_processing_time_ms": 12.8,
36         "uptime_seconds": 3600,
37         "messages_per_second": 0.14,
38         "processing_rate": 0.14,
39         "drop_rate_percent": 0.0,
40         "config": {
41             "server_ip": "127.0.0.1",
42             "server_port": 9001,
43             "topics": ["semantic_map"],
44             "file_prefix": "sem_",
45             "max_queue_size": 100,
46             "worker_thread_count": 4,
47             "queue_warning_threshold": 80
48         }
49     }
50 }
51 }
52 }
53 }
```

归一化规则:

下游 <code>status</code> 值	归一化结果
<code>running</code>	<code>running</code>
<code>stopped</code>	<code>stopped</code>

服务不可达	unknown
-------	---------

detail 顶层字段说明:

字段	类型	条件	说明
status	string	始终	running / stopped
timestamp	int	始终	Unix 时间戳（秒）
statistics	object	仅 running	运行统计信息

statistics 字段说明:

路径	类型	说明
total_images_saved	int	已保存图像总数
total_image_errors	int	图像处理错误总数
async_processing.containers_processed	int	已处理容器数
async_processing.containers_dropped	int	已丢弃容器数
async_processing.messages_dropped	int	异步处理链累计丢弃消息数
async_processing.dedup_storage_running	bool	去重存储线程是否运行
async_processing.queue_size	int	当前异步处理队列长度
async_processing.max_queue_size	int	异步处理队列上限
config.debug_mode	bool	是否调试模式
client_manager.running	bool	客户端管理器是否运行
client_manager.connected	bool	WebSocket 是否已连接
client_manager.total_messages_received	int	接收消息总数

<code>client_manager.total_messages_processed</code>	<code>int</code>	已处理消息总数
<code>client_manager.total_files_saved</code>	<code>int</code>	已保存文件总数
<code>client_manager.total_processing_errors</code>	<code>int</code>	处理错误总数
<code>client_manager.total_messages_dropped</code>	<code>int</code>	客户端层累计丢弃消息数
<code>client_manager.current_queue_size</code>	<code>int</code>	当前队列长度
<code>client_manager.queue_high_watermark</code>	<code>int</code>	队列历史峰值
<code>client_manager.queue_usage_percent</code>	<code>float</code>	当前队列使用率 (%)
<code>client_manager.avg_queue_wait_time_ms</code>	<code>float</code>	平均排队等待时间 (ms)
<code>client_manager.avg_processing_time_ms</code>	<code>float</code>	平均处理耗时 (ms)
<code>client_manager.uptime_seconds</code>	<code>int</code>	运行时长（仅运行中出现）
<code>client_manager.messages_per_second</code>	<code>float</code>	平均接收速率（仅运行中出现）
<code>client_manager.processing_rate</code>	<code>float</code>	平均处理速率（仅运行中出现）
<code>client_manager.drop_rate_percent</code>	<code>float</code>	消息丢弃率 (%)

注意: statistics 中部分字段为条件出现，客户端应按可选字段方式解析。

POST /v1/api/services/semantic/start

启动语义建图服务。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "success": true,
6      "message": "语义建图服务启动成功",
7      "status": "running"
8    }
9  }
```

已运行时的响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "success": true,
6      "message": "语义建图服务已经在运行中",
7      "status": "running"
8    }
9  }
```

错误:

code	说明
1005	语义建图服务不可达，或内部启动失败

POST /v1/api/services/semantic/stop

停止语义建图服务。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
```



```
3     "msg": "success",
4     "data": {
5         "success": true,
6         "message": "语义建图服务已停止",
7         "status": "stopped"
8     }
9 }
```

已停止时的响应:

代码块

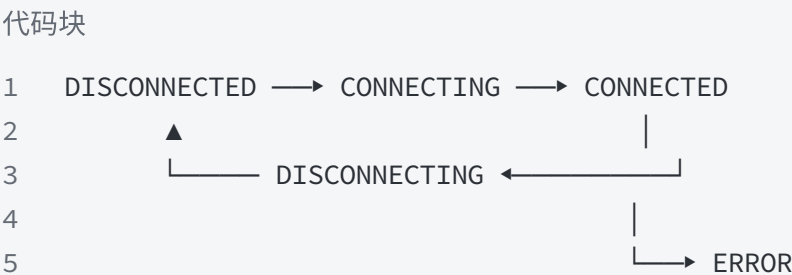
```
1  {
2      "code": 0,
3      "msg": "success",
4      "data": {
5          "success": true,
6          "message": "语义建图服务已经处于停止状态",
7          "status": "stopped"
8      }
9  }
```

错误: 1005 — 语义建图服务不可达。

4.3 运动控制 (Motion)

运动控制服务管理机器人适配器的生命周期，支持多种机器人类型（如 go2、m20 等）。同一时刻只能有一个适配器处于激活状态。

适配器状态机:



状态	说明
DISCONNECTED	无适配器运行

CONNECTING	正在启动适配器并连接机器人
CONNECTED	已连接，可正常控制
DISCONNECTING	正在停止适配器
ERROR	适配器异常退出或故障

下游错误码（motion 子服务内部 code）：

code	名称	说明
0	NONE	成功
1	UNKNOWN_ROBOT	未知机器人类型
2	TARGET_UNAVAILABLE	适配器服务不可达
3	BUSY	正在执行切换/停止流程
4	CONNECT_FAILED	连接失败
5	DISCONNECT_FAILED	断开失败
6	PRECONDITION_REQUIRED	前置条件不满足

Orsus 会将上述错误封装在标准 APIResponse 中返回，code 为 1005（下游服务错误）或透传下游数据。

GET /v1/api/services/motion/status

查询运动控制服务归一化状态。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "running",
6      "detail": {
7        "state": "CONNECTED",
8        "active_adapter": "go2",
9        "busy": false,
```

```
10     "last_code": "NONE",
11     "last_message": "adapter started and connected",
12     "last_detail": "adapter_type=go2; pid=12345",
13     "adapters": [
14         {
15             "robot_type": "go2",
16             "registered": true,
17             "reachable": true,
18             "available": true,
19             "detail": "G02 health ok"
20         }
21     ]
22 }
23 }
24 }
```

归一化规则:

下游 state 值	归一化结果
CONNECTED	running
DISCONNECTED / DISCONNECTING	stopped
CONNECTING / ERROR	unknown
服务不可达	unknown

detail 字段说明:

字段	类型	说明
state	string	适配器状态: DISCONNECTED / CONNECTING / CONNECTED / DISCONNECTING / ERROR
active_adapter	string	当前激活的适配器类型名，无则为 "unknown"
busy	bool	是否正在处理启动/停止操作
last_code	string	最近一次操作结果码名称 (NONE / UNKNOWN_ROBOT / BUSY 等)
last_message	string	最近一次操作摘要

last_detail	string	最近一次操作详情
adapters	array	所有注册适配器的健康状态列表

adapters 元素字段:

字段	类型	说明
robot_type	string	机器人类型
registered	bool	是否已注册
reachable	bool	健康检查是否可达
available	bool	是否可正常工作
detail	string	健康检查详情

POST /v1/api/services/motion/start

启动指定机器人适配器。启动后服务会等待健康检查通过并自动执行连接。

请求体:

代码块

```
1  {
2    "adapter_type": "go2"
3  }
```

字段	类型	必填	说明
adapter_type	string	是	机器人适配器类型，可通过 /services/motion/adapters 查询合法值

成功响应:

代码块

```
1  {
2    "code": 0,
```

```
3     "msg": "success",
4     "data": {
5         "active_adapter": "go2",
6         "state": "CONNECTED",
7         "detail": "adapter_type=go2; pid=12345"
8     }
9 }
```

data 字段说明:

字段	类型	说明
active_adapter	string	操作后激活的适配器，无则为 "unknown"
state	string	操作后的适配器状态
detail	string	操作详情

错误:

code	说明
1001	adapter_type 参数缺失
1005	运动控制服务不可达，或下游返回错误（如 BUSY / CONNECT_FAILED）

下游错误示例（适配器正忙）：

代码块

```
1  {
2      "code": 0,
3      "msg": "success",
4      "data": {
5          "code": 3,
6          "msg": "adapter transaction in progress",
7          "data": {
8              "active_adapter": "unknown",
9              "state": "CONNECTING",
10             "detail": "state=kStarting"
11         }
12     }
```

```
13 }
```

当下游返回非零 code 但 HTTP 200 时，Orsus 仍以 code: 0 透传完整的下游响应。客户端应检查 data 内部的 code 字段判断实际成功/失败。

POST /v1/api/services/motion/stop

停止当前运行中的机器人适配器。停止流程依次执行: safe_stop → disconnect → 停止进程。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "active_adapter": "unknown",
6      "state": "DISCONNECTED",
7      "detail": "adapter_type=go2"
8    }
9  }
```

错误: 1005 — 运动控制服务不可达。

业务规则: 若系统正在执行切换流程，下游返回 code: 3 (BUSY)。若无运行中适配器，仍返回成功。

GET /v1/api/services/motion/system_info

获取当前激活适配器上报的机器人系统信息。system_info 字段结构由具体适配器决定。

请求参数: 无

成功响应 (go2 适配器示例) :

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "active_adapter": "go2",
6      "state": "CONNECTED",
```

```
7      "system_info": {
8          "connected": true,
9          "network_interface": "eth0",
10         "has_low_state": true,
11         "has_sport_state": true,
12         "low": {
13             "battery_soc": 85,
14             "power_v": 28.5
15         },
16         "sport": {
17             "mode": 1,
18             "gait_type": 1,
19             "velocity": [0.5, 0.0, 0.0],
20             "yaw_speed": 0.1
21         }
22     }
23 }
24 }
```

无运行中适配器时的响应:

代码块

```
1  {
2      "code": 0,
3      "msg": "success",
4      "data": {
5          "active_adapter": "unknown",
6          "state": "DISCONNECTED",
7          "system_info": null
8      }
9  }
```

下游 code: 6 (PRECONDITION_REQUIRED) 表示无运行中适配器。

data 字段说明:

字段	类型	说明
active_adapter	string	当前激活适配器，无则"unknown"
state	string	当前适配器状态
system_info	object null	

		机器人系统信息，无适配器时为 null
--	--	------------------------

system_info 字段说明（go2 适配器）：

字段	类型	说明
connected	bool	机器人是否已连接
network_interface	string	网络接口名
has_low_state	bool	是否有底层状态数据
has_sport_state	bool	是否有运动状态数据
low.battery_soc	int	电池电量 (%)
low.power_v	float	电源电压 (V)
sport.mode	int	运动模式
sport.gait_type	int	步态类型
sport.velocity	float[3]	速度向量 [vx, vy, vz]
sport.yaw_speed	float	偏航角速度

注意: system_info 的具体结构取决于适配器类型，不同机器人返回的字段可能不同。客户端应按动态对象方式解析。

GET /v1/api/services/motion/adapters

获取服务端配置中已启用的适配器类型列表。客户端在调用 /start 前可查询此接口获取合法的 adapter_type 取值范围。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "enabled_adapter_types": ["go2", "m20"]
6    }
}
```



```
7 }
```

字段	类型	说明
<code>enabled_adapter_types</code>	<code>string[]</code>	已启用适配器类型列表（按字母序排列）

列表内容由服务启动时加载的配置决定，运行期间不会变化。

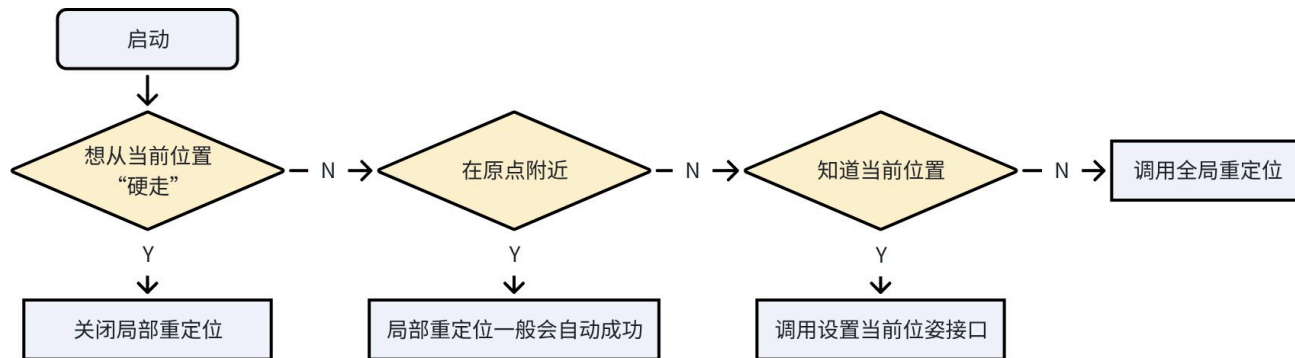
5. 导航 (Navigation)

导航功能说明

导航主要有自主规划和打点导航两个模式，特点如下：

模式	对应接口	特点	备注
自主规划	- 绕障单点导航 - 绕障多点导航	- 根据地图和指定目标点自主规划路径，目标点在可通行范围内均有效。 - 根据在线点云数据避开动态障碍物。	- 目前仅支持差速运动模型，全向和阿克曼底盘暂不支持。 - 仅支持机器人向前运动，后退行为被禁止。
打点导航	停障单点导航	根据机器人当前位置和目标位置间直线轨迹运动，运动轨迹完全可预测。	- 支持差速运动模式和全向模式（可以通过 <code>holonomic</code> 参数调整），阿克曼底盘暂不支持。 - 当选择全向运动模式时，根据目标点选择，可以支持向后运动。

重定位功能说明



- 当前手动位置设置支持 4 自由度位置，即 x，y，z 和绕 z 轴的旋转。
- 若机器人重定位时，传感器启动位置和建图时传感器启动位置的水平面角度有较大偏差，有较大概率会导致手动设置重定位失败。

5.1 导航容器生命周期

POST /v1/api/nav/container/start

启动导航容器（NavStack）。

请求体（可选）：

代码块

```
1  {
2    "bringup_mode": "default",
3    "scene_name": "office_floor1",
4    "use_relocalization": true,
5    "use_sim_time": false,
6    "use_online_map": false
7  }
```

字段	类型	必填	默认值	说明
bringup_mode	string	否	""	启动模式
scene_name	string	否	""	地图场景名
use_relocalization	bool	否	false	是否启用重定位
use_sim_time	bool	否	false	是否使用仿真时间
use_online_map	bool	否	false	是否使用在线地图

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

错误:

code	说明
2001	导航容器启动失败
1005	下游服务不可用

POST /v1/api/nav/container/stop

停止导航容器。**异步操作**——立即返回，后台执行。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

客户端应轮询 GET /nav/container/status 确认容器已停止。

GET /v1/api/nav/container/status

查询导航容器当前运行状态。

请求参数: 无

成功响应:

代码块

```
2      "code": 0,
3      "msg": "success",
4      "data": {
5          "running": true,
6          "status": "running",
7          "container_id": "abc123def456"
8      }
9  }
```

ContainerState 字段说明:

字段	类型	说明
running	bool	容器是否正在运行
status	string	状态: running / stopped / not_found / unknown
container_id	string	Docker 容器 ID

5.2 导航状态查询

POST /v1/api/nav/navigation_status

查询当前导航状态。

请求体: 无

成功响应:

代码块

```
1  {
2      "code": 0,
3      "msg": "success",
4      "data": {
5          "status": "navigating",
6          "relocalization": "active"
7      }
8  }
```

NavStatus 字段说明:

字段	类型	说明
status	string	导航状态
relocalization	string	重定位状态

POST /v1/api/nav/navigation_task_status

查询当前导航任务状态。

请求体: 无

成功响应: 同 navigation_status ，返回 NavStatus 对象。

5.3 导航控制

/nav/missions 是面向业务侧的导航任务接口，内部提交到统一任务队列。它比底层 /nav/start_navigation、/nav/start_path_navigation 更接近业务语义，支持标准导航、路径导航、巡逻、定点导航和复杂任务。

POST /v1/api/nav/missions

创建并提交导航任务。接口采用快速失败检测：任务提交后短暂等待，若立即失败则返回错误，否则返回 mission_id 。

请求体公共字段:

字段	类型	必填	说明
mode	string	是	任务模式：standard / route / direct / complex
frame_id	string	否	坐标系，默认由下游任务处理
target	object	视模式而定	单目标位姿
waypoints	array	视模式而定	路径点列表
cycles	int	否	route 模式循环次数，默认 1
default_params	object	否	complex 模式默认参数
steps	array	complex 必填	complex 模式步骤列表

mode	适用场景
standard	单点绕障导航（自动避让障碍物）
direct	单点停障导航（遇障停车，适合自动门、电梯口）
route	多点连续路径，按顺序经过所有路径点
complex	复合任务：将"绕障/停障/原地等待/原地旋转/速度切换"任意编排

standard / direct 示例:

代码块

```
1  {
2    "mode": "standard",
3    "frame_id": "map",
4    "target": {
5      "x": 1.0,
6      "y": 2.0,
7      "theta": 0.0
8    }
9  }
```

route 示例:

代码块

```
1  {
2    "mode": "route",
3    "frame_id": "map",
4    "cycles": 1,
5    "waypoints": [
6      { "x": 1.0, "y": 2.0, "theta": 0.0 },
7      { "x": 3.0, "y": 4.0, "theta": 1.57 }
8    ]
9  }
```

complex 示例:

代码块

```
1  {
2    "mode": "complex",
```

```
3  "frame_id": "map",
4  "default_params": {
5    "speed": 0.5
6  },
7  "steps": [
8    {
9      "type": "navigate",
10     "target": { "x": 1.0, "y": 2.0, "theta": 0.0 },
11     "on_failure": "abort"
12   },
13   {
14     "type": "wait",
15     "wait_seconds": 3
16   },
17   {
18     "type": "rotate",
19     "theta": 1.57,
20     "on_failure": "retry",
21     "retry": 2
22   }
23 ]
24 }
```

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "mission_id": "task-001",
6      "status": "pending"
7    }
8  }
```

校验规则:

mode	规则
standard	target 必填，绕障导航
direct	target 必填，停障导航
route	waypoints 必填， cycles <= 0 时按 1 处理

<code>complex</code>	<code>steps</code> 必填；不支持顶层 <code>target</code> / <code>waypoints</code> / <code>cycles</code>
----------------------	--

`complex.steps[].type` 支持：

type	必填字段	说明
<code>navigate</code>	<code>target</code>	导航到目标点
<code>fixed_point</code>	<code>target</code>	定点导航
<code>rotate</code>	<code>theta</code>	原地旋转
<code>wait</code>	<code>wait_seconds</code>	等待指定秒数

`on_failure` 可选值为 `abort`、`skip`、`retry`；使用 `retry` 时 `retry` 必须大于等于 1。

GET /v1/api/nav/missions/{id}

查询导航任务状态。

路径参数:

参数	类型	必填	说明
<code>id</code>	<code>string</code>	是	创建任务返回的 <code>mission_id</code>

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "mission_id": "task-001",
6      "mode": "complex",
7      "status": "running",
8      "current_step": 1,
9      "total_steps": 3
10   }
11 }
```


`current_step` 和 `total_steps` 仅 `complex` 模式可能出现。

DELETE /v1/api/nav/missions/{id}

取消导航任务。

路径参数:

参数	类型	必填	说明
<code>id</code>	<code>string</code>	是	创建任务返回的 <code>mission_id</code>

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

若任务已处于 `completed`、`failed` 或 `cancelled` 终态，接口按幂等成功处理。

POST /v1/api/nav/pause_navigation

暂停当前导航任务。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

POST /v1/api/nav/resume_navigation

恢复已暂停的导航任务。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

POST /v1/api/nav/get_plan_path

获取当前规划路径的全局坐标点序列。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "global": [
6        [1.0, 2.0],
7        [1.5, 2.3],
8        [2.0, 2.8]
9      ]
10   }
11 }
```

PathData 字段说明:

字段	类型	说明
global	float[n][2]	全局路径坐标序列，每个元素为 [x, y]

5.4 重定位

POST /v1/api/nav/relocalization_toggle

开启或关闭重定位功能。该接口要求导航容器处于运行中

请求体:

代码块

```
1  {
2    "enable": true
3  }
```

字段	类型	必填	说明
enable	bool	是	true 启用重定位， false 关闭

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

POST /v1/api/nav/set_relocalization_pose

手动设置机器人初始位姿（用于重定位校正）。该接口要求导航容器处于运行中

请求体:

代码块

```
1  {
2    "x": 1.0,
3    "y": 2.0,
4    "theta": 0.5
5  }
```

字段	类型	必填	说明
x	float	是	X 坐标（米）
y	float	是	Y 坐标（米）
theta	float	是	朝向角度（弧度）

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```

5.5 导航参数

平台服务对上游保留两类接口：

- 兼容接口： `/get_nav_params`、 `/set_nav_params`、 `/get_saved_nav_params`，用于读取/保存 平台中的导航参数，并在导航服务就绪时同步到参数服务。
- 原生参数接口： `/params/list`、 `/params/get`、 `/params/set`、 `/params/health`，直接代理参数服务语义。

参数名使用冒号路径格式，例如 `navigation:free_navigation:linear_speed`。旧参数名仍做兼容映射：

旧参数名	新参数名
linear_speed	navigation:free_navigation:linear_speed
angular_speed	navigation:free_navigation:angular_speed
robot_footprint	robot:footprint
fixed_point_safety_distance	navigation:fixed_point_navigation:forward_safety_distance
holonomic	navigation:fixed_point_navigation:holonomic

current_map	general:map_name
-------------	------------------

可调外部参数

参数名	类型	说明	示例
robot:lidar_height	double	雷达相对底盘平面的安装高度，单位 m。范围 0.0 到 3.0。	0.6
robot:footprint	array	机器人水平面物理包络，多边形点列表，单位 m。至少 3 个点，必须包含原点且不能自交。	[[0.3,0.16], [0.3,-0.16], [-0.35,-0.16], [-0.35,0.16]]
navigation:free_navigation:min_obstacle_height	double	绕障模式障碍检测最低 z 高度，单位 m。	0.1
navigation:free_navigation:max_obstacle_height	double	绕障模式障碍检测最高 z 高度，单位 m。	1.0
navigation:free_navigation:linear_speed	double	绕障模式最大线速度，单位 m/s。	0.5
navigation:free_navigation:angular_speed	double	绕障模式最大角速度，单位 rad/s。	0.7
navigation:free_navigation:xy_goal_tolerance	double	绕障模式平面距离到达阈值，单位 m。	0.5
navigation:free_navigation:yaw_goal_tolerance	double	绕障模式朝向到达阈值，单位 rad。	0.2
navigation:free_navigation:safety_distance	double	绕障模式障碍物到机器人包络边界的安全距离，单位 m。	0.2
navigation:fixed_point_navigation:max_linear_speed	double	停障/定点导航最大线速度，单位 m/s。	0.5

<code>navigation:fixed_point_navigation:max_angular_speed</code>	<code>double</code>	停障/定点导航最大角速度，单位 rad/s。	<code>0.7</code>
<code>navigation:fixed_point_navigation:holonomic</code>	<code>bool</code>	停障/定点导航是否使用全向控制。	<code>true</code>
<code>navigation:fixed_point_navigation:xy_goal_tolerance</code>	<code>double</code>	停障/定点导航平面距离到达阈值，单位 m。	<code>0.5</code>
<code>navigation:fixed_point_navigation:yaw_goal_tolerance</code>	<code>double</code>	停障/定点导航朝向到达阈值，单位 rad。	<code>0.2</code>
<code>navigation:fixed_point_navigation:lateral_safety_distance</code>	<code>double</code>	停障模式机器人包络左右侧额外通道余量，单位 m。	<code>0.2</code>
<code>navigation:fixed_point_navigation:forward_safety_distance</code>	<code>double</code>	停障模式前向停车距离余量，单位 m。	<code>0.2</code>

POST /v1/api/nav/get_nav_params

批量查询导航参数。该接口兼容旧参数名，响应按请求中的参数名返回。

请求体:

代码块

```
1  {
2    "names": [
3      "navigation:free_navigation:linear_speed",
4      "navigation:fixed_point_navigation:holonomic"
5    ]
6  }
```

字段	类型	必填	说明
<code>names</code>	<code>string[]</code>	是	要查询的参数名列表

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "navigation:free_navigation:linear_speed": {
6        "success": true,
7        "value": 0.5,
8        "message": "parameter read"
9      },
10     "navigation:fixed_point_navigation:holonomic": {
11       "success": true,
12       "value": true,
13       "message": "parameter read"
14     }
15   }
16 }
```

ParamResult 字段说明:

字段	类型	说明
success	bool	参数是否查询成功
value	any	参数值
message	string	失败时的错误信息

POST /v1/api/nav/get_saved_nav_params

批量查询 system-controller DB 中已保存的导航参数。该接口不要求导航容器正在运行；如果某参数未保存，则该参数返回 `success=false`。

请求体:

代码块

```
1  {
2    "names": ["navigation:free_navigation:linear_speed"]
3  }
```

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "navigation:free_navigation:linear_speed": {
6        "success": true,
7        "value": 0.5,
8        "message": ""
9      }
10   }
11 }
```

POST /v1/api/nav/set_nav_params

批量保存并设置导航参数。后端会先写入 system-controller DB；如果导航容器和参数服务已经就绪，会继续同步调用参数服务。若导航未运行，则参数只保存到 DB，后续导航启动后自动回放。

请求体（任意键值对）：

代码块

```
1  {
2    "robot:lidar_height": 0.62,
3    "navigation:free_navigation:linear_speed": 0.6,
4    "navigation:fixed_point_navigation:holonomic": true
5  }
```

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "robot:lidar_height": {
6        "success": true,
7        "message": "parameter applied",
8        "value": 0.62
9      },
10     "navigation:free_navigation:linear_speed": {
```



```
11     "success": true,
12     "message": "parameter applied",
13     "value": 0.6
14 },
15 "navigation:fixed_point_navigation:holonomic": {
16     "success": true,
17     "message": "parameter applied",
18     "value": true
19 },
20 "offline_save": {
21     "success": true,
22     "message": "offline config saved"
23 }
24 }
25 }
```

ParamSetResult 字段说明:

字段	类型	说明
success	bool	参数是否设置成功
message	string	失败时的错误信息
value	any	参数服务返回的实际值（可能省略）

POST /v1/api/nav/params/list

列出参数服务暴露的全部参数树。

请求体: 无。

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "parameters": {
6        "robot": {
7          "lidar_height": {
8            "access": "external",
```

```
9         "type": "double",
10        "value": 0.6,
11        "writable": true
12    }
13 },
14    "navigation": {
15        "free_navigation": {
16            "linear_speed": {
17                "access": "external",
18                "type": "double",
19                "value": 0.5,
20                "writable": true
21            }
22        }
23    }
24 }
25 }
26 }
```

POST /v1/api/nav/params/get

按参数名查询参数服务中的当前值。请求参数与 `/get_nav_params` 相同；响应在 `data.parameters` 中返回。

请求体:

代码块

```
1  {
2    "names": ["navigation:free_navigation:linear_speed"]
3  }
```

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5        "parameters": {
6            "navigation:free_navigation:linear_speed": {
7                "success": true,
8                "value": 0.5,
9                "message": "parameter read"
```

```
10     }
11     }
12 }
13 }
```

POST /v1/api/nav/params/set

直接调用参数服务设置参数，不写入 system-controller DB。前端需要持久化参数时应使用 `/set_nav_params`。

请求体:

代码块

```
1  {
2    "robot:lidar_height": 0.62,
3    "navigation:free_navigation:linear_speed": 0.6
4  }
```

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "results": {
6        "robot:lidar_height": {
7          "success": true,
8          "message": "parameter applied",
9          "value": 0.62
10       },
11       "navigation:free_navigation:linear_speed": {
12         "success": true,
13         "message": "parameter applied",
14         "value": 0.6
15       }
16     },
17     "offline_save": {
18       "success": true,
19       "message": "offline config saved"
20     }
21   }
```

POST /v1/api/nav/params/health

查询参数服务健康状态。

请求体: 无。

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "healthy": true
6    }
7  }
```

5.6 地图标注 (Landmarks)

地图标注支持两类对象：

- `point`：命名点位，`points` 必须包含 1 个点。
- `route`：命名路线，`points` 至少包含 2 个点。

GET /v1/api/nav/landmarks

列出指定地图场景下的命名点位与命名路线。

查询参数:

参数	类型	必填	说明
<code>scene_name</code>	<code>string</code>	是	地图场景名

POST /v1/api/nav/landmarks

保存命名点位或命名路线。

请求体:

代码块

```
1  {
2    "name": "front-desk",
3    "scene_name": "office_floor1",
4    "kind": "point",
5    "points": [
6      { "x": 1.0, "y": 2.0, "theta": 0.0 }
7    ]
8  }
```

PUT /v1/api/nav/landmarks/{id}

更新指定 ID 的地图标注。请求体结构同新增接口，路径参数 `id` 为标注 ID。

DELETE /v1/api/nav/landmarks/{id}

删除指定 ID 的地图标注。

POST /v1/api/nav/landmarks/{id}/start

按命名点位或命名路线启动导航任务。点位会转成单点导航，路线会转成路径导航。

请求体（可选，仅路线使用）：

代码块

```
1  {
2    "cycles": 1
3  }
```

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "task_id": "task-002"
6    }
7  }
```

5.7 导航一键启动

POST /v1/api/nav/startup

编排启动运动控制、定位扫描、导航容器、已保存导航参数同步和初始位姿设置。任一步骤失败会返回 `CodeNavigation=2001`，`data.failed_step` 标明失败步骤；已完成步骤不会自动回滚。

请求体:

代码块

```
1  {
2    "motion": {
3      "adapter_type": "go2"
4    },
5    "pose": {
6      "x": 1.0,
7      "y": 2.0,
8      "theta": 0.0
9    },
10   "nav_container": {
11     "scene_name": "office_floor1",
12     "bringup_mode": "localization",
13     "use_relocalization": true,
14     "use_sim_time": false,
15     "use_online_map": false
16   }
17 }
```

字段	类型	必填	说明
<code>motion.adapter_type</code>	<code>string</code>	否	运动控制适配器类型；为空时使用第一个可用适配器
<code>pose.x</code> / <code>pose.y</code> / <code>pose.theta</code>	<code>float</code>	是	初始位姿
<code>nav_container.scene_name</code>	<code>string</code>	是	导航容器使用的地图场景名
<code>nav_container.bringup_mode</code>	<code>string</code>	否	导航 bringup 模式
<code>nav_container.use_relocalization</code>	<code>bool</code>	否	是否使用重定位
<code>nav_container.use_sim_time</code>	<code>bool</code>	否	是否使用仿真时间

nav_container.use_online_map	bool	否	是否使用在线地图
------------------------------	------	---	----------

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "steps_completed": ["motion", "scan", "nav_container", "nav_params",
6        "set_pose"],
7      "motion": { "adapter_type": "go2" },
8      "pose": { "x": 1.0, "y": 2.0, "theta": 0.0 },
9      "nav_container": { "scene_name": "office_floor1" }
10   }
```

6. 建图 (Mapping)

功能说明

- 建图命名时不能为纯数字，且不能包含除“-”，“_”外的特殊字符（如“!”，“”等符号可能会因此未知行为）
- 2D点云支持动态障碍物去除，但是其区域的雷达覆盖时间需要超过动态障碍物存在的时间方可去除。

POST /v1/api/mapping/container/start

启动建图容器（场景地图采集）。自动创建建图任务，采用**快速失败检测**（5 秒窗口，200ms 轮询）。

请求体:

代码块

```
1  {
2    "scene_name": "warehouse_zone_A"
3  }
```

字段	类型	必填	说明
scene_name	string	是	场景名称（用于输出目录命名）

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "task_id": "t-20260410-003"
6    }
7  }
```

错误:

code	说明
1001	scene_name 参数缺失
3001	建图容器启动失败

POST /v1/api/mapping/container/stop

停止建图容器。取消当前建图任务，触发地图自动保存。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": null
5  }
```


GET /v1/api/mapping/container/status

查询建图容器当前运行状态。

请求参数: 无

成功响应: 同导航容器的 `ContainerState` 结构。

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "running": true,
6      "status": "running",
7      "container_id": "xyz789"
8    }
9  }
```

POST /v1/api/mapping/status

查询当前建图进度状态（代理容器内部 API）。

请求体: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "mapping",
6      "perception_available": true,
7      "map_available": false,
8      "points_collected": 15234,
9      "scene_name": "warehouse_zone_A",
10     "output_folder": "/data/maps/warehouse_zone_A"
11   }
12 }
```

MappingStatus 字段说明:

--	--	--

字段	类型	说明
status	string	建图状态
perception_available	bool	感知模块是否就绪
map_available	bool	地图数据是否可用
points_collected	int	已采集点云数量
scene_name	string	当前场景名
output_folder	string	输出目录路径

GET /v1/api/mapping/health

查询建图容器健康状态。

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "healthy": true
6    }
7  }
```

7. 地图资源 (Maps)

GET /v1/api/maps

获取设备上已有地图列表，支持按名称前缀过滤。

查询参数:

参数	类型	必填	说明
scene_name	string	否	地图名称前缀过滤

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "total": 2,
6      "maps": [
7        {
8          "name": "office_floor1",
9          "path": "/data/maps/office_floor1",
10         "size": 1048576,
11         "file_count": 5,
12         "created_time": 1712736000.0,
13         "modified_time": 1712736000.0,
14         "resolution": 0.05,
15         "origin": [-10.0, -10.0, 0.0]
16       }
17     ]
18   }
19 }
```

MapInfo 字段说明:

字段	类型	说明
name	string	地图名称
path	string	地图目录绝对路径
size	int	目录总大小（字节）
file_count	int	目录内文件数
created_time	float	创建时间（Unix 时间戳）
modified_time	float	最后修改时间（Unix 时间戳）
resolution	float	地图分辨率（米/像素）
origin	float[]	地图原点坐标 [x, y, theta]

DELETE /v1/api/maps/{map_name}

删除指定地图及其全部文件。

路径参数:

参数	类型	必填	说明
map_name	string	是	地图名称

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "msg": "success"
6    }
7  }
```

PUT /v1/api/maps/{map_name}

更新指定地图的 PGM 栅格文件。

Content-Type: multipart/form-data

路径参数:

参数	类型	必填	说明
map_name	string	是	地图名称

表单字段:

字段	类型	必填	说明
pgm_file	file	是	PGM 格式地图文件

成功响应:

代码块

```
2     "code": 0,
3     "msg": "success",
4     "data": null
5 }
```

错误: 1001 — pgm_file 字段缺失。

GET /v1/api/maps/{map_name}/files

获取指定地图目录内所有文件的元信息。

路径参数:

参数	类型	必填	说明
map_name	string	是	地图名称

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "map_name": "office_floor1",
6      "total": 3,
7      "files": [
8        {
9          "name": "map.pgm",
10         "path": "map.pgm",
11         "full_path": "/data/maps/office_floor1/map.pgm",
12         "size": 524288,
13         "created_time": 1712736000.0,
14         "modified_time": 1712736000.0,
15         "mime_type": "image/x-portable-graymap"
16       }
17     ]
18   }
19 }
```

FileInfo 字段说明:

字段	类型	说明
name	string	文件名
path	string	相对路径
full_path	string	绝对路径
size	int	文件大小（字节）
created_time	float	创建时间（Unix 时间戳）
modified_time	float	修改时间（Unix 时间戳）
mime_type	string	MIME 类型

GET /v1/api/maps/{map_name}/files/{file_path}

下载指定地图目录内的单个文件（二进制流）。

Content-Type: application/octet-stream

路径参数:

参数	类型	必填	说明
map_name	string	是	地图名称
file_path	string	是	文件相对路径（支持嵌套目录）

成功响应: 文件二进制内容。

GET /v1/api/maps/{map_name}/archive

下载整个地图目录的 ZIP 压缩包。

Content-Type: application/zip

路径参数:

参数	类型	必填	说明
map_name	string	是	地图名称

成功响应: ZIP 文件二进制内容（附带 `Content-Disposition: attachment; filename="{map_name}.zip"` 头）。

8. 日志管理 (Logs)

管理设备上的系统日志文件。接口同时注册在 `/logs` 和 `/ops/logs` 两个路径下。

GET /v1/api/logs/list

查询日志文件列表。

别名路径: GET /v1/api/logs/、GET /v1/api/ops/logs/list、GET /v1/api/ops/logs/

请求参数: 无

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "items": [
6        {
7          "name": "edge-core-2026-04-10.log",
8          "size": 102400,
9          "mod_time": "2026-04-10T10:00:00Z"
10       }
11     ]
12   }
13 }
```

GET /v1/api/logs/{filename}

读取指定日志文件内容。

别名路径: GET /v1/api/ops/logs/{filename}

路径参数:

参数	类型	必填	说明
filename	string	是	日志文件名

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "filename": "edge-core-2026-04-10.log",
6      "content": "[2026-04-10 10:00:00] INFO  server started on :8899\n..."
7    }
8  }
```

错误:

code	说明
1001	文件名不合法
1002	文件不存在

DELETE /v1/api/logs/{filename}

删除指定日志文件。

别名路径: DELETE /v1/api/ops/logs/{filename}

路径参数:

参数	类型	必填	说明
filename	string	是	日志文件名

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "deleted": "edge-core-2026-04-10.log"
6    }
}
```



```
7 }
```

POST /v1/api/logs/actions/cleanup

清理过期日志文件。

别名路径: POST /v1/api/logs/cleanup、POST /v1/api/ops/logs/actions/cleanup、POST /v1/api/ops/logs/cleanup

请求体（可选）：

代码块

```
1 {
2   "max_age_days": 7
3 }
```

字段	类型	必填	默认值	说明
max_age_days	int	否	配置文件值	保留天数（0 表示使用系统配置默认值）

成功响应:

代码块

```
1 {
2   "code": 0,
3   "msg": "success",
4   "data": {
5     "removed": 3,
6     "max_age_days": 7
7   }
8 }
```

9. 交互与预留接口

POST /v1/api/interact/asr

Qwen-ASR 语音识别代理接口。客户端上传音频文件，Orsus 在服务端携带 DashScope API Key 调用上游，并将上游 SSE 流式响应转发给客户端。

接口类型: SSE 成功响应，JSON 错误响应。

Content-Type: 请求为 `multipart/form-data`，成功响应为 `text/event-stream`。

请求参数:

参数	位置	类型	必填	说明
<code>audio</code>	form-data	file	是	音频文件，推荐 16-bit PCM mono WAV
<code>language</code>	query	string	否	识别语种，默认 <code>zh</code>
<code>enable_itn</code>	query	bool	否	是否启用 ITN，默认 <code>false</code>

成功响应:

上游 SSE 字节流原样转发，客户端按 OpenAI 兼容 chat completion stream 解析 `choices[0].delta.content`。

错误响应:

code	HTTP 状态码	说明
<code>1001</code>	<code>400</code>	缺少音频字段、音频为空或参数非法
<code>1003</code>	<code>500</code>	DashScope API Key 未配置或请求构造失败
<code>1005</code>	<code>502</code>	上游 DashScope 不可达或返回错误

附录 A: 数据模型定义

LocalTask — 本地任务

字段	类型	说明
ID	string	任务唯一标识
Type	string	任务类型: navigation / mapping
Status	int	任务状态: 0 =pending / 1 =running / 2 =completed / 3 =failed / 4 =cancelled / 5 =stopping
Payload	string	任务参数 JSON 字符串
CreatedAt	string	创建时间 (ISO 8601)
UpdatedAt	string	最后更新时间 (ISO 8601)
StartedAt	string null	开始执行时间
EndedAt	string null	结束时间
ErrorMsg	string	失败原因 (成功时为空串)
ErrorCode	int	错误码 (保留, 暂固定为 0)
IdempotencyKey	string	幂等键
TimeoutSeconds	int	执行超时秒数 (0 =不限时)

SavedPath — 已保存路径

字段	类型	说明
id	int	路径 ID (自动生成)
name	string	路径名称
scene_name	string	关联地图场景名
waypoints	Waypoint[]	路径点列表
created_at	string	创建时间 (ISO 8601)

Waypoint — 路径点

--	--	--

字段	类型	说明
x	float	X 坐标（米）
y	float	Y 坐标（米）
theta	float	朝向角度（弧度）

ContainerInfo — 容器信息

字段	类型	说明
name	string	容器服务名称
service_class	string	服务分类: model / infra
role	string	服务角色
container_name	string	Docker 容器名
state	string	状态: UNKNOWN / STOPPED / HEALTHY / DEGRADED / FAILED
health	string	健康检查结果
critical	bool	是否关键服务
allow_manual_stop	bool	是否允许手动停止
mutex_group	string	互斥组标识

ContainerState — 容器运行状态（导航/建图）

字段	类型	说明
running	bool	容器是否运行中
status	string	状态: running / stopped / not_found / unknown
container_id	string	Docker 容器 ID

NavStatus — 导航状态

--	--	--

字段	类型	说明
status	string	导航状态
relocalization	string	重定位状态

MappingStatus — 建图状态

字段	类型	说明
status	string	建图状态
perception_available	bool	感知模块是否就绪
map_available	bool	地图数据是否可用
points_collected	int	已采集点云数
scene_name	string	场景名
output_folder	string	输出目录

MapInfo — 地图信息

字段	类型	说明
name	string	地图名称
path	string	目录绝对路径
size	int	目录总大小（字节）
file_count	int	文件数
created_time	float	创建时间（Unix 时间戳）
modified_time	float	修改时间（Unix 时间戳）
resolution	float	分辨率（米/像素）
origin	float[]	原点坐标 [x, y, theta]

MapFileSet — 地图文件集

字段	类型	说明
----	----	----

map_name	string	地图名称
total	int	文件总数
files	FileInfo[]	文件信息列表

FileInfo — 文件信息

字段	类型	说明
name	string	文件名
path	string	相对路径
full_path	string	绝对路径
size	int	文件大小（字节）
created_time	float	创建时间（Unix 时间戳）
modified_time	float	修改时间（Unix 时间戳）
mime_type	string	MIME 类型

ServiceStatus — 服务状态

字段	类型	说明
status	string	归一化状态: running / stopped / degraded / unknown
detail	object	原始详情（各服务结构不同）

ParamResult — 参数查询结果

字段	类型	说明
success	bool	是否成功
value	any	参数值
message	string	错误信息

ParamSetResult — 参数设置结果

字段	类型	说明
success	bool	是否成功
message	string	错误信息

LogEvent — 日志事件

字段	类型	说明
ID	int	事件序列号
Service	string	来源服务名
Level	string	日志级别
Context	string	日志上下文/模块
TS	string	事件时间（ISO 8601）
Msg	string	日志消息
Raw	string	原始日志行

附录 B: SSE 事件流协议

端点: GET /v1/api/systems/stream

连接方式:

代码块

```
1  const es = new EventSource("http://<host>:8899/v1/api/systems/stream");
2
3  es.addEventListener("health", (e) => {
4    const health = JSON.parse(e.data);
5    console.log("Health:", health);
6  });
7
8  es.addEventListener("robot", (e) => {
9    const robot = JSON.parse(e.data);
10   console.log("Robot:", robot);
```

```
11    });
```

行为说明:

1. 连接建立后立即推送当前 `health` 和 `robot` 值
2. 之后 `health` 每 5 秒推送一次, `robot` 每 10 秒推送一次
3. 客户端断开连接时服务端自动清理资源
4. 建议客户端实现自动重连 (浏览器 `EventSource` 已内置)

事件数据结构:

`health` 事件 — 对应 `GET /v1/api/systems/health` 返回的 `data` 字段:

代码块

```
1  {"status":"ok","uptime_seconds":3600,"timestamp":"2026-04-10T10:00:00Z"}
```

`robot` 事件 — 对应 `GET /v1/api/systems/robot` 返回的 `data` 字段:

代码块

```
1  {"robot_type":"go2","connected":true,"battery":85}
```

`battery` 字段可能为数字 (单电池)、数组 (多电池) 或 `null` (运动控制服务不可达)。

响应头:

代码块

```
1  Content-Type: text/event-stream
2  Cache-Control: no-cache
3  Connection: keep-alive
4  X-Accel-Buffering: no
```

附录 C: WebSocket 帧协议

端点: `GET /v1/api/terminal/ws`

所有 WebSocket 消息使用 **Binary** 帧类型, 首字节为帧类型标识:

帧类型定义

帧类型	字节值	方向	说明
Input	0x00	客户端 → 服务端	终端输入（键盘数据）
Resize	0x01	客户端 → 服务端	终端窗口调整
Output	0x02	服务端 → 客户端	终端输出
Error	0x03	服务端 → 客户端	错误消息

帧结构

Input 帧 (0x00):

代码块

```
1  [0x00] [data bytes...]
```

- data : 键盘输入的原始字节

Resize 帧 (0x01):

代码块

```
1  [0x01] [cols_hi] [cols_lo] [rows_hi] [rows_lo]
```

- cols : 终端列数（Big-Endian uint16）
- rows : 终端行数（Big-Endian uint16）

Output 帧 (0x02):

代码块

```
1  [0x02] [data bytes...]
```

- data : PTY 输出数据

Error 帧 (0x03):

代码块

```
1  [0x03] [UTF-8 error message bytes...]
```

- 错误消息示例: "session idle timeout", "shell exited with code 1"

客户端连接示例

代码块

```
1  const ws = new WebSocket("ws://<host>:8899/v1/api/terminal/ws?token=xxx");
2  ws.binaryType = "arraybuffer";
3
4  // 发送输入
5  function sendInput(data) {
6      const buf = new Uint8Array(1 + data.length);
7      buf[0] = 0x00;
8      buf.set(new TextEncoder().encode(data), 1);
9      ws.send(buf);
10 }
11
12 // 发送窗口大小
13 function sendResize(cols, rows) {
14     const buf = new ArrayBuffer(5);
15     const view = new DataView(buf);
16     view.setUint8(0, 0x01);
17     view.setUint16(1, cols);
18     view.setUint16(3, rows);
19     ws.send(buf);
20 }
21
22 // 接收消息
23 ws.onmessage = (e) => {
24     const data = new Uint8Array(e.data);
25     const type = data[0];
26     const payload = data.slice(1);
27
28     switch (type) {
29         case 0x02: // Output
30             terminal.write(new TextDecoder().decode(payload));
31             break;
32         case 0x03: // Error
33             console.error("Terminal error:", new TextDecoder().decode(payload));
34             break;
35     }
36 };
```

超时机制

- 空闲超时默认 **10 分钟**
- `Input` 和 `Resize` 帧均会重置空闲计时器

- 超时后服务端发送 Error 帧 `"session idle timeout"` 并关闭连接
-

健康探针

GET /healthz

独立于 API 前缀的健康探针端点，供 Kubernetes/Docker 等编排系统使用。

成功响应:

代码块

```
1  {
2    "code": 0,
3    "msg": "success",
4    "data": {
5      "status": "ok"
6    }
7  }
```